微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

附录A　在Windows系统下编译OpenJDK 6

这是本书第1版中介绍如何在Windows下编译OpenJDK 6的例子，里面的部分内容现在已经过时了（例如安装Plug部分），但对在Windows上构建安装环境和进行较老版本的OpenJDK编译还是有一定参考意义的，所以笔者并没有把它删除，而是挪到附录之中。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

A.1 获取JDK源码

首先确定要使用的JDK版本，OpenJDK 6和OpenJDK 7都是开源的，源码都可以在它们的主页（http://openjdk.java.net/）上找到。OpenJDK 6的源码其实是从OpenJDK 7的某个基线中引出的，然后剥离JDK 1.7相关的代码，从而得到一份可以通过TCK 6的JDK 1.6实现，因此直接编译OpenJDK 7会更加“原汁原味”一些，其实这两个版本的编译过程差异并不大。

获取源码有两种方式：一是通过Mercurial代码版本管理工具从Repository中直接取得源码（Repository地址：http://hg.openjdk.java.net/jdk7/jdk7），这是最直接的方式，从版本管理中看变更轨迹比看什么Release Note都来得实在，不过太麻烦了一些，尤其是Mercurial远不如SVN、ClearCase或CVS之类的版本控制工具那样普及；另外一种就是直接下载官方打包好的源码包了，可以从Source Releases页面（地址：http://download.java.net/openjdk/jdk7/）取得打包好的源码，一般来说大概一个月左右会更新一次，虽然不够及时，但的确方便了许多。笔者下载的是OpenJDK 7 Early Access Source Build b121版，2010年12月9日发布的，大概81.7MB，解压出来约308MB。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

A.2 系统需求

如果可能，笔者建议尽量在Linux或Solaris上构建OpenJDK，这要比在Windows平台上轻松许多，而且网上能找到的资料绝大部分都是在Linux上编译的。如果一定要在Windows平台上编译，建议读者认真阅读一下源码中的README-builds.html文档（无论在OpenJDK网站上还是在下载的源码包里面都有这份文档），因为编译过程中需要注意的细节非常多。虽然不至于像文档上所描述的“Building the source code for the JDK requires a high level of technical expertise.Sun provides the source code primarily for technical experts who want to conduct research（编译JDK需要很高的专业技术，Sun提供JDK源码是为了供技术专家进行研究之用）”那么夸张，但是如果读者是第一次编译，那在上面耗费一整天乃至更多的时间都很正常。

笔者在本次实战中演示的是在32位Windows 7平台下编译x86版的OpenJDK（也就是32位的JDK），如果需要编译x64版，那毫无疑问也需要一个64位的操作系统。另外编译涉及的所有文件都必须存放在NTFS格式的文件系统中，因为FAT32格式无法支持大小写敏感的文件名。官方文档上写道：编译至少需要512MB的内存和600MB的磁盘空间。如果读者耐心很好的话，512MB的内存也可以凑合使用，不过600MB的磁盘空间仅仅是指存放OpenJDK源码和相关依赖项的空间，要完成编译，600MB肯定是无论如何都不够的。这次实战中所下载的工具、依赖项、源码，全部安装、解压完成最少（“最少”是指只下载C++编译器，不下载VS的IDE）需要1GB的空间。

对系统的最后一点要求就是所有的文件，包括源码和依赖项目，都不要放在包含中文或空格的目录里面，这样做不是一定不可以，只是这样会为后续建立CYGWIN环境带来很多额外的工作。这是由于Linux和Windows的磁盘路径差别所导致的，我们也没有必要自己给自己找麻烦。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

A.3 构建编译环境

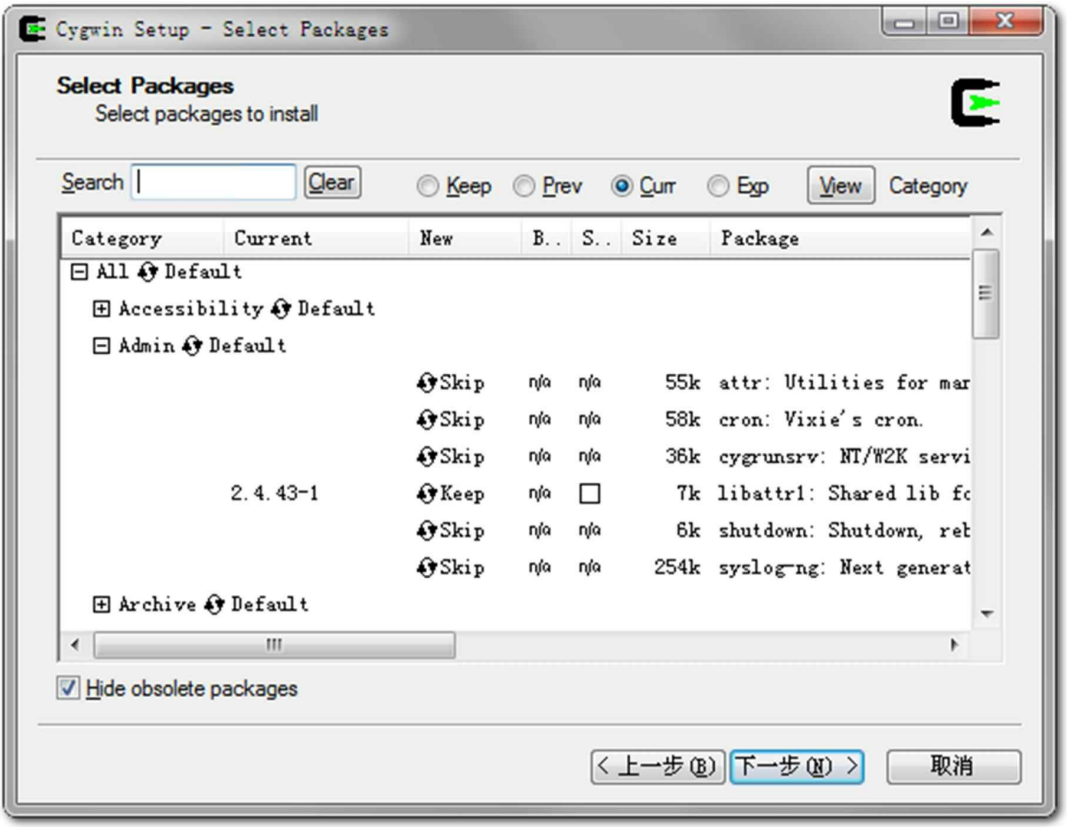
准备编译环境的第一步是安装一个CYGWIN[1]。这是一个在Windows平台下模拟Linux运行环境的软件，提供了一系列的Linux命令支持。需要CYGWIN的原因是，在编译中要使用GNU Make来执行Makefile文件（C/C++程序员肯定很熟悉，如果只使用Java，那把这个东西当成C++版本的ANT看待就可以了）。安装CYGWIN时不能直接默认安装，因为表A-1中所示的工具都不会进行默认安装，但又是编译过程中需要的，因此要在图A-1所示的安装界面中进行手工选择。

CYGWIN安装时的定制包选择界面如图A-1所示。

表A-1　需要手工选择安装的CYGWIN工具

文件名	分类	包	描述
ar.exe	Devel	binutils	The GNU assembler, linker and binary utilities
make.exe	Devel	make	The GNU version of the 'make' utility built for CYGWIN.
m4.exe	Interpreters	m4	GNU implementation of the traditional Unix macro processor
cpio.exe	Utils	cpio	A program to manage archives of files
gawk.exe	Utils	awk	Pattern-directed scanning and processing language
file.exe	Utils	file	Determines file type using 'magic' numbers
zip.exe	Archive	zip	Package and compress (archive) files
unzip.exe	Archive	unzip	Extract compressed files in a ZIP archive
free.exe	System	procps	Display amount of free and used memory in the system

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程



图A-1　CYGWIN安装界面

建立编译环境的第二步是安装编译器。JDK中最核心的代码（Java虚拟机及JDK中Native方法的实现等）是使用C++语言及少量的C语言编写的，官方文档中说它们的内部开发环境是在Microsoft Visual Studio C++2003（VS2003）中进行编译的，同时也是在Microsoft Visual Studio C++2010（VS2010）中测试过的，所以最好只选择这两个编译器之一进行编译。如果选择VS2010，那么要求在编译器之中已经包含了Windows SDK v 7.0a，否则可能还要自己去下载这个SDK，并且更新PlatformSDK目录。由于笔者没有购买Visual Studio 2010的IDE，所以仅下载了VS2010 Express中提取出来的C++编译器，这部分是免费的，但单独安装好编译器比较麻烦。建议读者选择使用整套Visual Studio C++2010或Visual Studio C++2010 Express版进行编译。

需要特别注意的一点是：CYGWIN和VS2010安装之后都会在操作系统的PATH环境变量中写入自己的bin目录路径，必须检查并保证VS2010的bin目录在CYGWIN的bin目录之前，因为这两个软件的bin目录之中各自都有一个连接器“link.exe”，但是只有VS2010中的连接器可以完成OpenJDK的编译。

准备JDK编译环境的第三步就是下载一个已经编译好的JDK。这听起来也许有点滑稽——要用鸡蛋孵小鸡还真得必须先养一只母鸡呀？但仔细想想，其实这个步骤很合理：因为JDK包含的各个部分

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

（Hotspot、JDK API、JAXWS、JAXP……）有的是使用C++编写的，而更多的代码则是使用Java自身实现的，因此编译这些Java代码需要用到一个可用的JDK，官方称这个JDK为Bootstrap JDK。而编译OpenJDK 7的话，Bootstrap JDK必须使用JDK6 Update 14或之后的版本，笔者选用的是JDK6 Update 21。

最后一个步骤是下载一个Apache ANT，JDK中Java代码部分都是使用ANT脚本进行编译的，ANT版本要求在1.6.5以上，这部分是Java的基础知识，对本书的读者来说应该没有难度，笔者不再详述。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

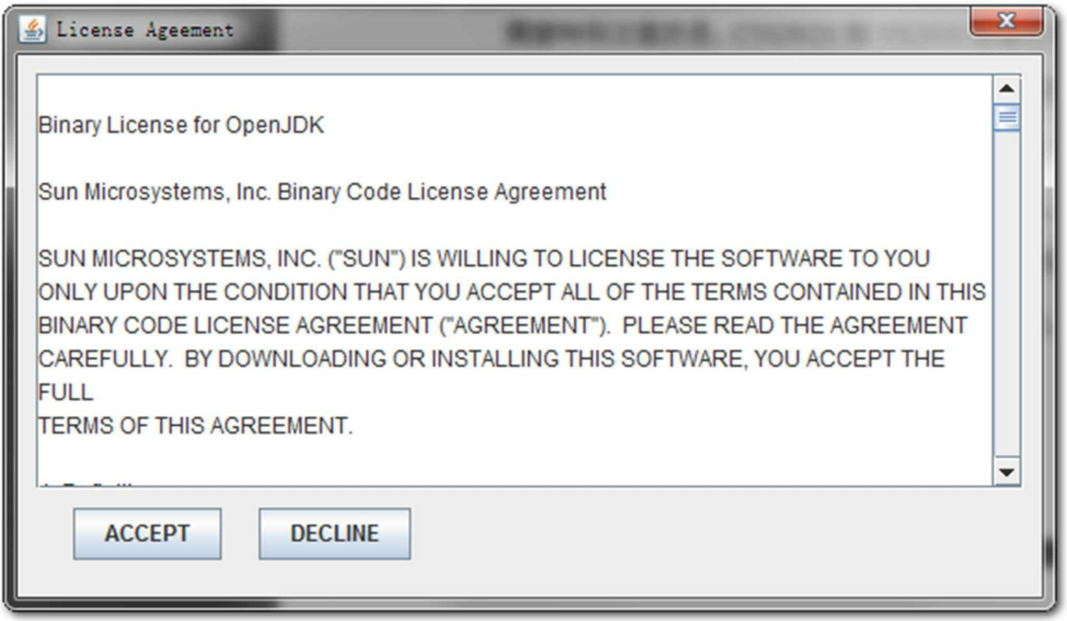
A.4 准备依赖项

前面说过，OpenJDK中开放的源码并没有达到100%，还有极少量的无法开源的产权代码存在。OpenJDK承诺日后将逐步使用开源实现来替换掉这部分产权代码，但至少在今天，编译JDK还需要这部分闭源包，官方称之为“JDK Plug”[2]，它们从前面的Source Releases页面就可以下载到。Windows平台的JDK Plug是以Jar包的形式提供的，通过下面这条命令可以安装它：

```
java -jar jdk-7-ea-plug-b121-windows-i586-09_dec_2010.jar
```

运行后将会显示图A-2所示的协议，点击“ACCEPT”接受协议，然后把Plug安装到指定目录即可。安装完毕后建立一个环境变量ALT_BINARY_PLUGS_PATH，变量值为此JDK Plug的安装路径，后面编译程序时需要用到它。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程



图A-2　JDK Plug安装协议

除了要用到JDK Plug外，编译时还需要引用JDK的运行时包，这是编译JDK中用Java代码编写的那部分所需要的，如果仅仅是想编译一个HotSpot虚拟机则可以不用。官方文档把这部分称为Optional Import JDK，可以直接使用前面Bootstrap JDK的运行时包。我们需要建立一个名为ALT_JDK_IMPORT_PATH的环境变量指向JDK的安装目录。

然后，安装一个大于2.3版的FreeType[3]，这是一个免费的字体渲染库，JDK的Swing部分和JConsole这类工具会用到它。安装好后建立两个环境变量ALT_FREETYPE_LIB_PATH和ALT_FREETYPE_HEADERS_PATH，分别指向FreeType安装目录下的bin目录和include目录。另外还有一点是官方文档没有提到但必须要做的事情，那就是把FreeType的bin目录加入PATH环境变量中。

接着，下载Microsoft DirectX 9.0 SDK（Summer 2004），安装后大约有298MB，在微软官方网站上搜索一下就可以找到下载地址，它是免费的。安装后建立环境变量ALT_DXSDK_PATH指向DirectX 9.0 SDK的安装目录。

最后，寻找一个名为MSVCR100.DLL的动态链接库，如果读者在前面安装了全套的Visual Studio 2010，那这个文件在本机就能找到，否则上网搜索一下也能找到单独的下载地址，大概有744KB。建立环境变量ALT_MSVCRNN_DLL_PATH指向这个文件所在的目录。如果读者选择的是VS2003，这个文件名应当为MSVCR73.DLL，应该在很多软件中都包含有这个文件，如果找不到的话，前面下载的Bootstrap JDK的bin目录中应该也有一个，直接拿来用吧。

微信 642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

A.5 进行编译

现在需要下载的编译环境和依赖项目都准备齐全了，最后我们还需要对系统做一些设置以便编译能够顺利通过。

首先执行VS2010中的VCVARS32.BAT，这个批处理文件的目的主要是设置INCLUDE、LIB和PATH这几个环境变量，如果和笔者一样只是下载了编译器，则需要手工设置它们。各个环境变量的设置值可以参考下面给出的代码清单A-1中的内容。批处理运行完之后建立ALT_COMPILER_PATH环境变量，让Makefile知道在哪里可以找到编译器。

再建立ALT_BOOTDIR和ALT_JDK_IMPORT_PATH两个环境变量指向前面提到的JDK 1.6的安装目录。建立ANT_HOME指向Apache ANT的安装目录。建立的环境变量很多，为了避免遗漏，笔者写了一个批处理文件以供读者参考，如代码清单A-1所示。

代码清单A-1　环境变量设置

```
SET ALT_BOOTDIR=D:/_DevSpace/JDK 1.6.0_21
SET ALT_BINARY_PLUGS_PATH=D:/jdkBuild/jdk7plug/openjdk-binary-plugs
SET ALT_JDK_IMPORT_PATH=D:/_DevSpace/JDK 1.6.0_21
SET ANT_HOME=D:/jdkBuild/apache-ant-1.7.0
SET ALT_MSVCRNN_DLL_PATH=D:/jdkBuild/msvcr100
SET ALT_DXSDK_PATH=D:/jdkBuild/msdxsdk
SET ALT_COMPILER_PATH=D:/jdkBuild/vcpp2010.x86/bin
SET ALT_FREETYPE_HEADERS_PATH=D:/jdkBuild/freetype-2.3.5-1-bin/include
SET ALT_FREETYPE_LIB_PATH=D:/jdkBuild/freetype-2.3.5-1-bin/bin

SET INCLUDE=D:/jdkBuild/vcpp2010.x86/include;D:/jdkBuild/vcpp2010.x86/sdk/Include;%INCLUDE%
SET LIB=D:/jdkBuild/vcpp2010.x86/lib;D:/jdkBuild/vcpp2010.x86/sdk/Lib;%LIB%
SET LIBPATH=D:/jdkBuild/vcpp2010.x86/lib;%LIB%
SET PATH=D:/jdkBuild/vcpp2010.x86/bin;D:/jdkBuild/vcpp2010.x86/dll/x86;D:/Software/OpenSource/cygwin/bir
```

最后还需要进行两项调整，官方文档没有说明这两项，但是必须要做完才能保证编译过程的顺利完成：一是取消环境变量JAVA_HOME，这点很简单；另外一项是尽量在英文的操作系统上编译。估计大部分读者会感到比较为难吧？如果不能在英文的系统上编译就把系统的文字格式调整为“英语（美国）”，在控制面板-区域和语言选项的第一个页签中可以设置。如果这个设置还不能更改就建立一个BUILD_CORBA环境变量，将其值设置为false，取消编译CORBA部分。否则Java IDL（idlj.exe）为*.idl文件生成CORBA适配器代码的时候会产生中文注释，而这些中文注释会因为字符集的问题而导致编译失败。

完成了上述烦琐的准备工作之后，我们终于可以开始编译了。进入控制台（Cmd.exe）后运行刚才准备好的设置环境变量的批处理文件，然后输入bash进入Bourne Again Shell环境（习惯sh或ksh的读者请自便）。如果JDK的安装源码中存在jdk_generic_profile.sh这个Shell脚本，先执行它，笔者下载的OpenJDK 7 B121版没有这个文件了，所以直接输入make sanity来检查我们前面所做的设置是否全部正确。如果一切顺利，几秒钟之后会有类似代码清单A-2所示的输出。

代码清单A-2　make sanity检查

```
D:\jdkBuild\openjdk7>bash
```

微信642620018，获取慕课网、极客时间、腾讯课堂、网易云课堂全套课程

```
bash-3.2$ make sanity
cygwin warning:
    MS-DOS style path detected: C:/Windows/system32/wscript.exe
    Preferred POSIX equivalent is: /cygdrive/c/Windows/system32/wscript.exe
    CYGWIN environment variable option "nodosfilewarning" turns off this warning.
    Consult the user's guide for more details about POSIX paths:
        http://cygwin.com/cygwin-ug-net/using.html#using-pathnames
( cd  ./jdk/make && \
```

……因篇幅关系，中间省略了大量的输出内容……

```
OpenJDK-specific settings:
    FREETYPE_HEADERS_PATH = D:/jdkBuild/freetype-2.3.5-1-bin/include
        ALT_FREETYPE_HEADERS_PATH = D:/jdkBuild/freetype-2.3.5-1-bin/include
    FREETYPE_LIB_PATH = D:/jdkBuild/freetype-2.3.5-1-bin/bin
        ALT_FREETYPE_LIB_PATH = D:/jdkBuild/freetype-2.3.5-1-bin/bin

OPENJDK Import Binary Plug Settings:
    IMPORT_BINARY_PLUGS = true
    BINARY_PLUGS_JARFILE = D:/jdkBuild/jdk7plug/openjdk-binary-plugs/jre/lib/rt-closed.jar
        ALT_BINARY_PLUGS_JARFILE =
    BINARY_PLUGS_PATH = D:/jdkBuild/jdk7plug/openjdk-binary-plugs
        ALT_BINARY_PLUGS_PATH = D:/jdkBuild/jdk7plug/openjdk-binary-plugs
    BUILD_BINARY_PLUGS_PATH = J:/re/jdk/1.7.0/promoted/latest/openjdk/binaryplugs
        ALT_BUILD_BINARY_PLUGS_PATH =
    PLUG_LIBRARY_NAMES =

Previous JDK Settings:
    PREVIOUS_RELEASE_PATH = USING-PREVIOUS_RELEASE_IMAGE
        ALT_PREVIOUS_RELEASE_PATH =
    PREVIOUS_JDK_VERSION = 1.6.0
        ALT_PREVIOUS_JDK_VERSION =
    PREVIOUS_JDK_FILE =
        ALT_PREVIOUS_JDK_FILE =
    PREVIOUS_JRE_FILE =
        ALT_PREVIOUS_JRE_FILE =
    PREVIOUS_RELEASE_IMAGE = D:/_DevSpace/JDK 1.6.0_21
        ALT_PREVIOUS_RELEASE_IMAGE =
Sanity check passed.
```

Makefile的Sanity检查过程输出了编译所需的所有环境变量，如果看到“Sanity check passed.”则说明检查过程通过了，可以输入“make”执行整个Makefile，然后就去喝个下午茶再回来了。笔者Core i5/4GB RAM的机器编译整个JDK大概需要半个小时的时间。如果失败则需要根据系统输出的失败原因，回头再检查一下对应的设置。最好在下一次编译之前先执行“make clean”来清理掉上次编译遗留的文件。

编译完成之后，打开OpenJDK源码下的build目录，看看是不是已经有一个编译好的JDK在那里等着了？执行一下“java-version”，看到以自己机器命名的JDK了吧？很有成就感吧？

[1] CYGWIN下载地址：http://www.cygwin.com/。
[2] 在2011年，JDK Plug已经不再需要了，但在笔者写本次实战使用的2010年12月9日发布的OpenJDK b121版时还是需要这些JDK Plug的。
[3] FreeType主页：http://www.freetype.org/。